Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. doi: 10.18637/jss.v000.i00

CausalMixGPD: Bayesian NP Model with Optional Heavy-Tail Augmentaion and Causal Extensions

Arnab Aich
Florida State University

Abstract

Heavy-tailed outcomes arise in many scientific domains where rare events dominate risk, cost, or policy decisions. Classical parametric models often fit the center of the distribution at the expense of tail misspecification, while purely nonparametric methods typically lack principled extrapolation in regions with sparse data. Extreme value theory (EVT) provides asymptotic justification for generalized Pareto distributions (GPDs) for threshold exceedances, whereas Bayesian nonparametric Dirichlet process mixtures (DPMs) provide flexible modeling for heterogeneous bulk behavior. This article introduces `CausalMixGPD`, an `R` package implementing a Bayesian semiparametric framework that combines conditional DPMs for the bulk distribution with covariate-dependent GPD tails for exceedances, with posterior predictive inference for densities, distribution functions, quantiles, survival probabilities, and tail risks. The framework is conditional by construction; unconditional models arise as a special case when covariates are omitted. The package supports both Chinese restaurant process (CRP) and stick-breaking (SB) representations of the Dirichlet process, multiple kernel families for bulk modeling, and an extension to causal inference via treatment-specific conditional distributions that enable inference on distributional and tail-sensitive causal estimands. In addition, the package includes a model-based clustering wing (bulk-only and spliced), supporting covariate-dependent mixture weights, covariate-dependent component parameters, or both, with clustering summaries based on posterior co-clustering probabilities.

Keywords: Bayesian nonparametrics; Dirichlet Process Mixture; Generalized Pareto Distribution; Extreme Value Theory; Causal Inference; `R`; `nimble`.

1. Introduction

Causal inference is often summarized through average effects, but many applications require distributional comparisons between treatment regimes. In clinical trials, investigators may

want to understand how a treatment changes outcomes across the outcome scale, including effects among individuals in the upper tail where events are rare but consequences can be substantial. This shift from mean-based to distribution-based causal questions makes causal analysis tightly linked to how well we can estimate (and predict) outcome distributions under competing treatment assignments.

A growing R ecosystem supports Bayesian causal inference through flexible outcome modeling and posterior uncertainty quantification. For example, bcf implements Bayesian Causal Forests for binary treatments and continuous outcomes, providing posterior inference for both average and heterogeneous treatment effects (bcf 2024). The package bartCause builds causal estimators around Bayesian additive regression trees, using BART as a regression engine for treatment-response modeling and causal effect estimation (bar 2025; Hill 2011). Complementing tree-based approaches, BCEE implements a Bayesian model-averaging strategy for causal effect estimation with support for binary or continuous exposures and outcomes (BCE 2023; Talbot, Lefebvre, Atherton, and Chiu 2015). A recurring practical difficulty in quantile-based causal reporting is quantile crossing issue, i.e., when two quantile curves for different indices cross each other. To avoid that issue this package (CausalMixGPD) directly model outcome densities and derives quantiles and other functionals from the posterior predictive distributions.

In the context of density estimation and conditional density estimation, DPMs provide a versatile non parametric solution. It can identify multimodality and latent heterogeneity without specifying number of latent mixing components. In expreimental framework R is well supported by Bayesian nonparametric mixture modeling software based on DP ideas. For example, dirichletprocess provides a flexible interface for specifying DPMs and fitting them by posterior simulation, yielding posterior inference for mixture weights, component allocations, and predictive densities under user-chosen kernels (dir 2023). The package DPpackage offers a broad suite of Bayesian semi- and nonparametric models in R, including DP-based formulations for marginal and conditional densities, regression functions, and model-based predictive distributions (Jara, Hanson, Quintana, Müller, and Rosner 2011). More recently, BNPmix emphasizes practical Bayesian nonparametric mixture approaches, providing posterior inference for clustering structure and density/regression estimation under DP and Pitman–Yor-type mixture specifications (Corradin, Canale, and Nipoti 2021). However, DPMs are suitable for modeling the bulk of the distribution but do not explicitly model tail behavior, which can lead to poor estimation and prediction in the tails when data are sparse or a heavy tail is present. mMoreover, these packages use a single kernel family for the entire distribution, which may not be ideal when the bulk and tail exhibit different characteristics.

Bayesian implementations of EVT are available in R when posterior uncertainty quantification for tail quantitites are required. For example, evdbayes provides Bayesian inference for standard extreme-value models via posterior simulation, enabling posterior and posterior-predictive summaries for extreme-event quantities (evd 2025). For peaks-over-threshold (POT) approaches, the package POT implements a focused collection of tools for GPD modeling and threshold-based tail analysis (POT 2024). Finally, for extreme quantile estimation and rare-event tail summaries, extremefit provides a published software implementation targeted specifically at extreme-quantile estimation (Durrieu, Grama, Jaunatre, Pham, and Tricot 2018).

This article introduces, an R package (CausalMixGPD) that connects Bayesian nonparametric bulk modeling with extreme-value tail modeling, while also supporting robust causal analyses

in presence of covariates(confounders). The package provides three simultaneous capabilities:

- **Bayesian Non-Parametric density modeling and prediction with optional GPD tail:** CausalMixGPD fits DPMs using a user-chosen kernel below a certain threshold (bulk part), and can optionally splice a generalized Pareto tail above the threshold (tail part) to enable the estimation of tail quantities and posterior predictive inference for rare-event summaries.
- **Model-based clustering:** a dedicated clustering interface that supports three setups where mixture weights, parameters and both depends on covariates and returns clustering summaries via posterior similarity matrices and Dahl least-squares partitions.
- **Causal estimation and prediction for RCTs and observational studies:** This package also supports causal analyses by modeling treatment-specific potetntial outcome densities and reporting contrasts as functionals of the posterior over densities.

Taken together, CausalMixGPD offers a coherent workflow for density estimation, prediction, and distributional causal reporting when tail behavior is scientifically central. The package is implemented in R with computationally intensive components coded in C++ and integrated via the `nimble` framework for flexible model specification and efficient posterior simulation (nim 2025). The remainder of this article is organized as follows: Section 2 provides background on the key modeling components, including DPMs, GPDs, and the spliced bulk-tail construction. Section 3 gives an overview of the package's main functions and user interface. Section 5 and Section 4 illustrates the package's capabilities through applications to causal inference and clustering, respectively. Finally, Section 6 concludes with a discussion of limitations and future directions.

2. Background and Preliminaries

This package treats conditional outcome distributions as the fundamental inferential object. All summaries reported later are functionals of estimated conditional distributions, including conditional quantiles, tail probabilities, and distributional causal contrasts.

Let the outcome–covariate pair be defined by

$$(Y, X) \in \mathbb{R} \times \mathbb{R}^p. \tag{1}$$

For a fixed covariate value x, the conditional cumulative distribution function (CDF) and density are

$$F(y \mid x) = \Pr\,(Y \leq y \mid X = x), \qquad f(y \mid x) = \frac{\partial}{\partial y} F(y \mid x), \tag{2}$$

and the conditional quantile function is defined by

$$Q(\tau \mid x) = \inf\Big\{y \in \mathbb{R} :\, F(y \mid x) \geq \tau\Big\}, \qquad 0 < \tau < 1. \tag{3}$$

2.1. DPM Model for Conditional Densities

I model the bulk ($Y < u$) conditional density of Y given $X = x$ using a DPM regression model. Let $k(\cdot \mid x; \theta)$ be a user-chosen kernel family indexed by parameter θ. The hierarchical specification is

$$Y \mid X = x, \theta \sim k(\cdot \mid x; \theta), \qquad \theta \mid H \sim H, \qquad H \mid \kappa, H_0 \sim \mathrm{DP}(\kappa, H_0), \tag{4}$$

where H is an unknown mixing distribution on the kernel-parameter space, H_0 is a base measure, and $\kappa > 0$ is a concentration parameter (Ferguson 1973; Lo 1984).

Marginalizing H yields an (almost surely) discrete mixing distribution and an infinite mixture representation:

$$f_{\mathrm{DP}}(y \mid x) = \int k(y \mid x; \theta)\, dH(\theta) = \sum_{j=1}^{\infty} w_j\, k(y \mid x; \theta_j), \qquad w_j \geq 0, \qquad \sum_{j=1}^{\infty} w_j = 1. \tag{5}$$

A constructive representation for the weights is given by Sethuraman's stick-breaking construction (Sethuraman 1994):

$$w_1 = V_1, \qquad w_j = V_j \prod_{\ell<j} (1 - V_\ell), \qquad V_j \overset{\text{iid}}{\sim} \mathrm{Beta}(1, \kappa), \qquad \theta_j \overset{\text{iid}}{\sim} H_0. \tag{6}$$

An alternative marginal representation is obtained by integrating out H, which induces the Blackwell–MacQueen P'olya urn predictive scheme (often described via the Chinese restaurant process) (Blackwell and MacQueen 1973; Neal 2000):

$$\theta_{\text{new}} \mid \theta_{1:n} \sim \frac{\kappa}{\kappa + n} H_0(\cdot) + \frac{1}{\kappa + n} \sum_{m=1}^{n} \delta_{\theta_m}(\cdot). \tag{7}$$

Both stick-breaking (blocked/truncated) and allocation-based representations are widely used for posterior simulation in DPM models (Escobar and West 1995; Neal 2000). DPM modeling is flexible, it works without specifying the number of mixture components, and can capture complex distributional features such as multimodality, skewness, and heteroscedasticity by choosing an appropriate kernel family. This makes DPMs a natural choice for modeling the bulk of the conditional distribution, where data are abundant and complex features may be present. The choice of kernel family is user-controlled which I will discuss in Section 3.

2.2. GPD for Threshold Exceedances

At the tail region EVT provides a canonical approximation for exceedances above a high threshold. Let $u(x)$ be a (possibly covariate-dependent) high threshold and define the exceedance

$$Z = Y - u(x), \tag{8}$$

considered conditionally on the event $Y > u(x)$. The conditional excess distribution function is

$$F_u(z \mid x) = \Pr\left(Y - u(x) \leq z \mid Y > u(x), X = x\right), \qquad z \geq 0. \tag{9}$$

Under broad regularity conditions, $F_u(\cdot)$ is well-approximated by a GPD for sufficiently large u (Balkema and de Haan 1974; Pickands 1975; Coles 2001). With scale parameter $\sigma(x) > 0$

and shape parameter $\xi \in \mathbb{R}$, the CDF for tail excedances is given by

$$F_{\mathrm{GPD}}(y \mid u(x), \sigma(x)) = \begin{cases} 1 - \left(1 + \xi \dfrac{y - u(x)}{\sigma(x)}\right)^{-1/\xi}, & \xi \neq 0, \\ 1 - \exp\left(-\dfrac{y - u(x)}{\sigma(x)}\right), & \xi = 0, \end{cases} \qquad y > u(x), \tag{10}$$

with support $y \geq u(x)$ when $\xi \geq 0$, and $u(x) \leq y \leq u(x) - \sigma(x)/\xi$ when $\xi < 0$. The corresponding density is

$$f_{\mathrm{GPD}}(y \mid u(x), \sigma(x), \xi) = \begin{cases} \dfrac{1}{\sigma(x)} \left(1 + \xi \dfrac{y - u(x)}{\sigma(x)}\right)^{-1/\xi - 1}, & \xi \neq 0, \\ \dfrac{1}{\sigma(x)} \exp\left(-\dfrac{y - u(x)}{\sigma(x)}\right), & \xi = 0. \end{cases} \tag{11}$$

The associated GPD quantile function is given by

$$Q_{\mathrm{GPD}}(\tau \mid u(x), \sigma(x), \xi) = \begin{cases} u(x) - \sigma(x) \log(1 - \tau), & \xi = 0, \\ u(x) + \dfrac{\sigma(x)}{\xi}\left((1 - \tau)^{-\xi} - 1\right), & \xi \neq 0, \end{cases} \qquad 0 < \tau < 1. \tag{12}$$

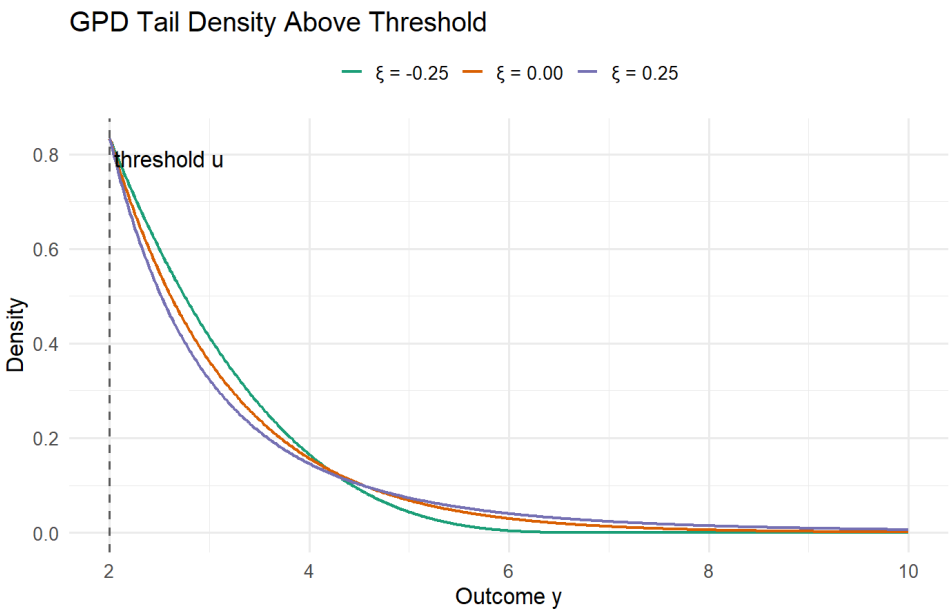

Figure 1: GPD tail density above threshold for multiple shape parameters.

Figure 1, captioned "GPD tail density above threshold for multiple shape parameters," illustrates the GPD density for different shape parameters ξ with a fixed scale parameter σ and threshold u. The shape parameter ξ controls the tail heaviness: $\xi < 0$ corresponds to a light tail with a finite upper bound, $\xi = 0$ corresponds to an exponential tail, and $\xi > 0$ corresponds to a heavy tail with infinite support. The GPD provides a flexible family for

modeling exceedances above a high threshold, making it a natural choice for tail modeling in the spliced construction described next.

2.3. Spliced DPM–GPD Conditional Distribution

DPM regression models Eq: (4) provide a flexible representation of the *bulk* of the conditional distribution of $Y \mid X = x$, however, in the far upper tail-where data are intrinsically sparse, EVT provides an parametric description of threshold exceedances via the GPD in Eq: (11). I therefore adopt a *spliced* (bulk–tail) construction that retains the DPM fit below a high threshold and replaces the upper tail by a GPD component (Frigessi, Haug, and Rue 2002; Behrens, Lopes, and Gamerman 2004; Coles 2001). Let

$$p_u(x) = F_{\mathrm{DP}}(u(x) \mid x; \Theta) \tag{13}$$

denote the bulk probability mass below a threshold ($u(x)$) under the DPM fit. The spliced conditional CDF is given as

$$F(y \mid x; \Theta, \Phi) = \begin{cases} F_{\mathrm{DP}}(y \mid x; \Theta), & y \le u(x), \\ p_u(x; \Theta) + \{1 - p_u(x; \Theta)\}\, F_{\mathrm{GPD}}(y \mid x; \Phi), & y > u(x), \end{cases} \tag{14}$$

where $F_{\mathrm{DP}}(\cdot)$ is the DPM CDF on the bulk region and $F_{\mathrm{GPD}}(\cdot)$ is the GPD CDF on the exceedance region (Section 2.2). The CDF (14) is continuous at $u(x)$ and yields a proper distribution. Differentiating (14) yields the spliced density

$$f(y \mid x; \Theta, \Phi) = \begin{cases} f_{\mathrm{DP}}(y \mid x; \Theta), & y \le u(x), \\ \{1 - p_u(x; \Theta)\}\, f_{\mathrm{GPD}}(y \mid x; \Phi), & y > u(x), \end{cases} \tag{15}$$

so the tail density is a *scaled* GPD density, with scaling ensuring normalization of $f(\cdot \mid x; \Theta, \Phi)$. For $y > u(x)$, the conditional survival function decomposes as

$$\Pr\,(Y > y \mid X = x; \Theta, \Phi) = \{1 - p_u(x; \Theta)\}\Big[1 - F_{\mathrm{GPD}}(y \mid x; \Phi)\Big]. \tag{16}$$

Quantiles follow by inversion of (14). For $\tau \in (0, 1)$,

$$Q(\tau \mid x; \Theta, \Phi) = \begin{cases} Q_{\mathrm{DP}}(\tau \mid x; \Theta), & \tau \le p_u(x; \Theta), \\ Q_{\mathrm{GPD}}(\tilde{\tau}(x; \Theta) \mid x; \Phi), & \tau > p_u(x; \Theta), \end{cases} \tag{17}$$

where the tail-rescaled probability level is

$$\tilde{\tau}(x; \Theta) = \frac{\tau - p_u(x; \Theta)}{1 - p_u(x; \Theta)} \in (0, 1), \tag{18}$$

and Q_{GPD} is given in (12). Thus, upper quantiles are determined jointly by the bulk exceedance probability $1 - p_u(x; \Theta)$ and the conditional exceedance distribution governed by Φ. The following figure illustrates the spliced density construction, showing the bulk DPM density, the GPD tail density, and the resulting spliced density.

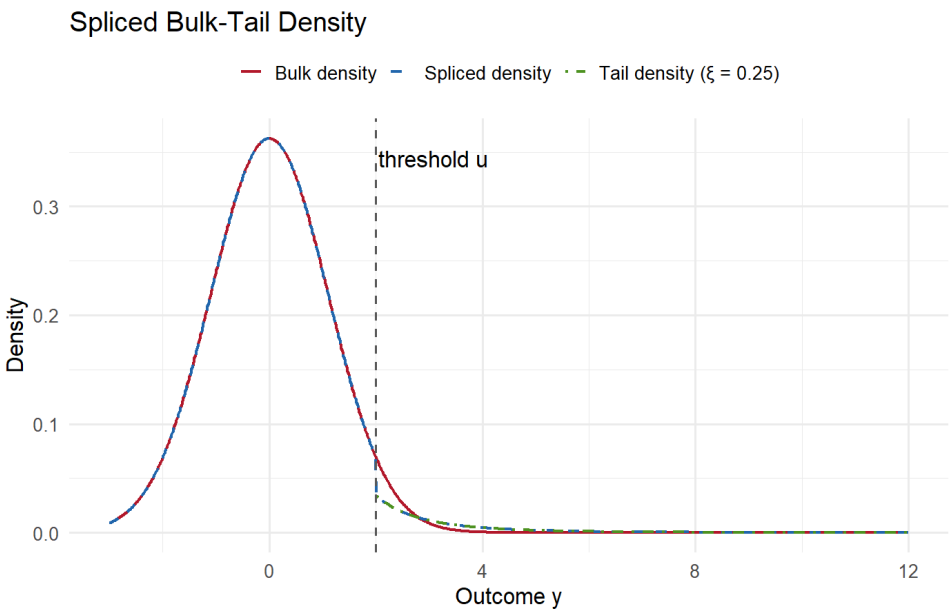

Figure 2: Illustration of the spliced bulk–tail density construction.

Figure 2, captioned "Illustration of the spliced bulk–tail density construction," visualizes the spliced bulk–tail density construction.

To make the splicing and downstream functionals explicit, we collect the unknown bulk regression parameters into Θ and the tail parameters into Φ. Specifically, the bulk model in (5) can be written as

$$f_{\mathrm{DP}}(y \mid x; \theta) = \sum_{j=1}^{\infty} w_j \, k(y \mid x; \theta_j(x)) \,, \qquad \Theta = \{\beta_j\}_{j \geq 1}, \tag{19}$$

where β_j denotes component-specific regression coefficients and

$$\theta_j(x) = g(x^\top \beta_j)$$

denotes the kernel parameter vector induced at covariate value x through the corresponding coefficients in Θ by a user chosen link function $g(\cdot)$. Mixture weights $\{w_j\}$ are generated by the DP backend (SB or CRP representation) which we will discuss in Section 3. It is to be noted that Θ may contain other kernel parameters that may not depend on covariates. As an example if the kernel is a Gaussian density, then Θ may contain regression coefficients for the mean (β_j^μ) and a variance parameter (σ_j) that does not depend on covariates.

For the tail block, let

$$\Phi = \{\beta_u, \beta_\sigma, \xi\}, \tag{20}$$

where $u(x)$ and $\sigma(x)$ are induced by covariates and tail-regression coefficients as

$$u(x) = g_u(x^\top \beta_u), \qquad \sigma(x) = g_\sigma(x^\top \beta_\sigma),$$

with $g_u(\cdot)$ and $g_\sigma(\cdot)$ being user-chosen link functions. Lastly, ξ is the tail shape parameter as discussed in Section 2.2. Thus, Θ and Φ are global parameter collections, while x enters

through these mappings. The package allows users to control which kernel and tail parameters vary with covariates (e.g., via link-based regression for selected parameters) and which are treated as fixed or purely a priori parameters (Section 3).

This spliced conditional distribution is the central predictive object used throughout the package. In the causal extension (Section 2.5), the same construction is applied separately to treatment-specific conditional distributions $F_a(\cdot \mid x)$, enabling causal contrasts based on densities, quantiles, and tail risks rather than only mean-scale effects. For datasets with moderate to light tail behavior, users can choose to fit a DPM without splicing by using appropriate settings for the tail (See Section 3.2.1).

2.4. Clustering under predictor-dependent Bayesian nonparametric mixtures

A common Bayesian nonparametric approach to conditional density modeling represents the collection $\{f(\cdot \mid x) : x \in \mathcal{X}\}$ through predictor-indexed random probability measures. Within the Dependent Dirichlet Process (DDP) framework of MacEachern (1999, 2000), one specifies a family $\{G_x : x \in \mathcal{X}\}$ of random mixing measures whose dependence across x induces flexible borrowing of information in $f(\cdot \mid x)$.

A generic mixture representation is

$$f(y \mid x) \;=\; \int k(y \mid x, \theta)\; dG_x(\theta), \tag{21}$$

where $k(\cdot \mid x, \theta)$ is a kernel and G_x is a predictor-indexed mixing measure. When G_x is discrete (as under stick-breaking constructions), latent parameters exhibit ties, inducing a random partition of observations. Introduce allocation variables z_i and component parameters $\{\theta_j\}_{j\geq 1}$ such that

$$y_i \mid z_i, \{\theta_j\}, x_i \sim k(\,\cdot \mid x_i, \theta_{z_i})\,. \tag{22}$$

Clustering is then characterized by the posterior distribution of the induced partition $\Pi = \{z_1, \ldots, z_n\}$ and summarized in a label-invariant manner using co-clustering functionals (Binder 1978; Dahl 2006).

In the clustering component considered here, we allow three forms of predictor dependence, each corresponding to a particular construction of G_x (equivalently, predictor dependence in the weights and/or atoms of a stick-breaking representation). In all cases, the conditional density admits the series form

$$f(y \mid x) \;=\; \sum_{j=1}^{\infty} w_j(x)\, k(y \mid x, \theta_j)\,, \tag{23}$$

with $w_j(x) \geq 0$ and $\sum_{j=1}^{\infty} w_j(x) = 1$ for all x.

- **Weight dependence only:** predictors modulate mixture weights while atoms are shared across x. Predictors may affect only the mixing weights, with atoms shared across x:

$$f(y \mid x) \;=\; \sum_{j=1}^{\infty} w_j(x)\, k(y \mid \theta_j)\,. \tag{24}$$

This specification is appropriate when covariates modulate subpopulation prevalence while within-subpopulation distributions remain stable. A common approach uses predictor-dependent stick-breaking, where stick lengths are transformed regressions in x; the logistic stick-breaking process constructs these lengths via logistic regressions and maps them to weights (Ren, Du, Carin, and Dunson 2011). Such models can be viewed as practical DDP instances in which dependence is introduced through smoothly varying weights (MacEachern 2000; Quintana, Müller, Jara, and MacEachern 2022).

- **Atom dependence only:** predictors modulate component-specific kernel parameters while weights are shared across x Alternatively, predictors may enter through component-specific kernel parameters while the weights remain global:

$$f(y \mid x) \;=\; \sum_{j=1}^{\infty} w_j \, k(y \mid x, \theta_j)\,. \tag{25}$$

  Here, θ_j typically indexes cluster-specific regression parameters (with appropriate link functions to enforce constraints), so that clusters correspond to distinct regression surfaces while w_j retains the interpretation of global prevalence. This structure aligns with DDP constructions that share weights while allowing predictor-indexed atoms or atom transformations (MacEachern 1999, 2000), and is closely related to ANOVA-type dependent priors for structured collections of random measures (De Iorio, Müller, Rosner, and MacEachern 2004).

- **Weight and atom dependence:** predictors modulate both mixture weights and component-specific kernel parameters The most flexible specification allows predictors to influence both prevalence and within-cluster behavior:

$$f(y \mid x) \;=\; \sum_{j=1}^{\infty} w_j(x) \, k(y \mid x, \theta_j)\,. \tag{26}$$

  This corresponds to DDP-type models in which both selection probabilities and component-specific conditional structure vary with predictors (Quintana *et al.* 2022).

Posterior clustering summaries are reported using label-invariant functionals of Π. The posterior similarity matrix (PSM) has entries

$$S_{i\ell} \;=\; \Pr(z_i = z_\ell \mid \text{data}), \qquad i, \ell = 1, \ldots, n, \tag{27}$$

estimated by Monte Carlo averages over MCMC output. A representative partition may be obtained by selecting the sampled partition whose adjacency matrix is closest to the PSM under squared loss (Dahl 2006), yielding a stable point estimate while retaining the PSM as the primary uncertainty summary.

2.5. Causal Inference: Notation, Identification, and Reported Estimands

I use the potential outcomes framework (Neyman 1990; Rubin 1974; Imbens and Rubin 2015). Let A denote a binary treatment indicator, let X denote observed pre-treatment covariates, and let Y denote the observed outcome. Each unit has two potential outcomes, $Y(1)$ and $Y(0)$, and the observed outcome satisfies

$$Y = A\,Y(1) + (1 - A)\,Y(0). \tag{28}$$

The treatment-specific conditional distribution functions are the fundamental inferential objects:

$$F_a(y \mid x) = \Pr\{Y(a) \le y \mid X = x\}, \qquad a \in \{0, 1\}. \tag{29}$$

In this package, the corresponding conditional densities are

$$f_a(y \mid x) = \frac{\partial}{\partial y} F_a(y \mid x) \tag{30}$$

where $f_a(y \mid x)$ assumes the spliced DPM structure in (15). Conditional quantiles are defined by inversion of the conditional CDF:

$$Q_a(\tau \mid x) = \inf\{y : F_a(y \mid x) \ge \tau\}, \qquad \tau \in (0, 1). \tag{31}$$

Conditional means are functionals of the conditional density:

$$\mu_a(x) = \mathrm{E}\{Y(a) \mid X = x\} = \int y\, f_a(y \mid x)\, dy. \tag{32}$$

Marginal (standardized) distributions are obtained by averaging conditional distributions over a covariate distribution. The package reports population-standardized and treated-standardized functionals:

$$F_a^{\mathrm{m}}(y) = \mathrm{E}\{F_a(y \mid X)\}, \qquad F_a^{\mathrm{t}}(y) = \mathrm{E}\{F_a(y \mid X) \mid A = 1\}. \tag{33}$$

The corresponding marginal quantiles are

$$Q_a^{\mathrm{m}}(\tau) = \inf\{y : F_a^{\mathrm{m}}(y) \ge \tau\}, \qquad Q_a^{\mathrm{t}}(\tau) = \inf\{y : F_a^{\mathrm{t}}(y) \ge \tau\}. \tag{34}$$

Marginal means are

$$\mu_a^{\mathrm{m}} = \mathrm{E}\{\mu_a(X)\}, \qquad \mu_a^{\mathrm{t}} = \mathrm{E}\{\mu_a(X) \mid A = 1\}. \tag{35}$$

Identification of these quantities depends on the study design. In randomized experiments, randomization together with positivity implies exchangeability between treatment assignment and potential outcomes (Imbens and Rubin 2015). In observational studies, we rely on the following conditions (Rosenbaum and Rubin 1983; Imbens and Rubin 2015):

- **SUTVA:** no interference between units and no hidden versions of treatment.
- **Consistency:** if $A = a$, then the observed outcome equals the corresponding potential outcome, $Y = Y(a)$.
- **Conditional ignorability:** treatment is as-if randomized given covariates,

$$(Y(0), Y(1)) \;\perp\; A \;\mid\; X. \tag{36}$$

- **Overlap (positivity):** each covariate profile has positive probability of receiving either treatment,

$$0 < \Pr(A = 1 \mid X = x) < 1 \tag{37}$$

  for covariate values of interest.

Under these conditions, the treatment-specific conditional distributions are identified from the observed data:

$$F_a(y \mid x) = \Pr\left(Y \leq y \mid A = a, X = x\right), \qquad a \in \{0, 1\}. \tag{38}$$

In observational settings, we use the propensity score (PS) to support the design stage (Rosenbaum and Rubin 1983). The PS is the treatment assignment probability given covariates:

$$\rho(x) = \Pr\left(A = 1 \mid X = x\right). \tag{39}$$

A key property of the true propensity score is that conditioning on it balances covariates across treatment groups:

$$A \perp X \mid \rho(X). \tag{40}$$

With estimated scores this balancing property is approximate in finite samples. Following the two-stage strategy described in the thesis, we first estimate $\rho(x)$ using only (A, X) and then augment the outcome model covariates with the estimated score (Rosenbaum and Rubin 1983):

$$\widehat{\rho}(x) \approx \rho(x), \qquad r(x) = \left(x^{\mathsf{T}}, \widehat{\rho}(x)\right)^{\mathsf{T}}. \tag{41}$$

The conditional outcome distributions are then modeled separately within each treatment arm as functions of the augmented predictor $r(x)$, and causal contrasts are computed as functionals of the estimated treatment-specific distributions.

The package reports the following causal estimands:

- **Average Treatment Effect (ATE):**

$$\mathrm{ATE} = \mu_1^{\mathrm{m}} - \mu_0^{\mathrm{m}}. \tag{42}$$

- **Average Treatment Effect on the Treated (ATT):**

$$\mathrm{ATT} = \mu_1^{\mathrm{t}} - \mu_0^{\mathrm{t}}. \tag{43}$$

- **Quantile Treatment Effect (QTE):** for any $\tau \in (0, 1)$ (Firpo 2007),

$$\mathrm{QTE}(\tau) = Q_1^{\mathrm{m}}(\tau) - Q_0^{\mathrm{m}}(\tau). \tag{44}$$

- **Quantile Treatment Effect on the Treated (QTT):** for any $\tau \in (0, 1)$,

$$\mathrm{QTT}(\tau) = Q_1^{\mathrm{t}}(\tau) - Q_0^{\mathrm{t}}(\tau). \tag{45}$$

- **Conditional Average Treatment Effect (CATE):**

$$\mathrm{CATE}(x) = \mu_1(x) - \mu_0(x). \tag{46}$$

- **Conditional Quantile Treatment Effect (CQTE):** for any $\tau \in (0, 1)$,

$$\mathrm{CQTE}(\tau \mid x) = Q_1(\tau \mid x) - Q_0(\tau \mid x). \tag{47}$$

This section established the main probabilistic and causal building blocks used in the remainder of the work. I first defined conditional distributional targets and introduced a DPM model to flexibly estimate bulk conditional outcome densities. I then reviewed generalized Pareto modeling for threshold exceedances and constructed a spliced DPM–GPD conditional distribution that preserves the estimated bulk while enabling principled tail extrapolation. Finally, we introduced the potential outcomes framework and the treatment-effect estimands reported in the package, showing how marginal and conditional effects can be obtained as functionals of treatment-specific conditional distributions (with propensity-score augmentation supporting confounding adjustment in observational studies). Subsequent chapters build on these preliminaries to specify the full hierarchical model, posterior computation, and empirical evaluation of tail-aware distributional causal effects.

3. Package Overview: Workflow, Model, and Inference

This section describes the main workflow provided by CausalMixGPD for one-arm conditional modeling and prediction when a generalized Pareto tail is spliced above a covariate-dependent threshold. The causal extension reuses the same modeling, estimation, and prediction machinery after introducing treatment-specific outcome models and a propensity score component; we defer those additions to the causal section.

3.1. Augmented DPM–GPD model, likelihood, and priors

Let the observed data be

$$\mathcal{D} = \{(y_i, x_i)\}_{i=1}^n,$$

```
data("nc_posX100_p3_k2", package = "CausalMixGPD")
dat <- data.frame(y = nc_posX100_p3_k2$y, nc_posX100_p3_k2$X)
```

with $x_i \in \mathbb{R}^p$ denoting the covariate vector (including an intercept if desired). The target conditional distribution $F(\cdot \mid x)$ and its spliced DPM–GPD construction are defined in Section 2, with the spliced CDF and density given in (14)–(15). Posterior computation in CausalMixGPD approximates the bulk DPM by a finite truncation with J mixture components,

$$f_{\mathrm{DP}}(y \mid x; \Theta) \;\approx\; \sum_{j=1}^{J} w_j \, k(y \mid x; \theta_j), \qquad j = 1, \ldots, J,$$

where j indexes mixture components, $w_j \geq 0$, and $\sum_{j=1}^{J} w_j = 1$. Kernel parameters can be constant or covariate-dependent through link specifications of the form

$$g(\psi_j(x)) = x^\top \beta_{j,\psi}, \tag{48}$$

where $g(\cdot)$ enforces parameter support (e.g., positive scale parameters).

```
mcmc_fixed <- list(
  niter = 200,
  nburnin = 50,
  thin = 1,
  nchains = 1,
  seed = 1
)
```

```
fit <- dpmgpd(
  formula = y ~ x1 + x2 + x3,
  data = dat,
  backend = "sb",
  kernel = "lognormal",
  components = 5,
  mcmc = mcmc_fixed
)
```

```
MixGPD fit | backend: Stick-Breaking Process | kernel:
Lognormal Distribution | GPD tail: TRUE
n = 100 | components = 5 | epsilon = 0.025
MCMC: niter=200, nburnin=50, thin=1, nchains=1
Fit
Use summary() for posterior summaries; plot() for
diagnostics; predict() for predictions.
```

For the spliced model, let $u(x)$ be the threshold, $\sigma(x) > 0$ the tail scale, and $\xi \in \mathbb{R}$ the tail shape (as in Section 2.3). Define $\delta_i = \mathbf{1}\{y_i > u(x_i)\}$. Using the spliced density (15) with the truncated bulk mixture, the likelihood is

$$L(\Theta, \Phi; \mathcal{D}) = \prod_{i=1}^{n} f(y_i \mid x_i; \Theta, \Phi), \tag{49}$$

where Θ collects the truncation-level bulk parameters $\{w_j, \theta_j\}_{j=1}^{J}$, and Φ collects the tail parameters governing $u(x)$, $\sigma(x)$, and ξ. Equivalently, the log-likelihood can be written as

$$\begin{aligned} \log L(\Theta, \Phi; \mathcal{D}) \;=\; & \sum_{i=1}^{n} (1 - \delta_i) \log f_{\mathrm{DP}}(y_i \mid x_i; \Theta) \\ & + \sum_{i=1}^{n} \delta_i \Big[\log\{1 - F_{\mathrm{DP}}(u(x_i) \mid x_i; \Theta)\} + \log f_{\mathrm{GPD}}(y_i \mid x_i; \Phi) \Big], \end{aligned} \tag{50}$$

where f_{GPD} is given in (11) and $F_{\mathrm{DP}}(\cdot \mid x)$ is the bulk mixture CDF induced by the truncated mixture.

Priors follow the decomposition implied by (6) and (7). Under the SB backend, the truncation-level weights are generated through Beta stick breaks (Section 2, (6)), which are represented in nimble using the Beta distribution in model code (see `dbeta`). Under the CRP backend, the allocation-based representation is implemented using the CRP distribution in nimble (see `dCRP`), matching the predictive representation in (7). Kernel parameters and tail parameters are assigned weakly informative priors, with regression coefficients $\beta_{j,\psi}$ arising through the link representation (48). The same link construction is used for covariate-dependent $u(x)$ and $\sigma(x)$ when those are modeled as functions of x.

3.2. MCMC algorithm, convergence diagnostics, and parameter extraction

Posterior simulation targets $p(\Theta, \Phi \mid \mathcal{D}) \propto L(\Theta, \Phi; \mathcal{D})\, p(\Theta, \Phi)$ using the likelihood (49) and the prior structure implied by the SB or CRP representation (Section 2, (6)–(7)). Given a model specification, CausalMixGPD runs MCMC through nimble (de Valpine, Turek, Paciorek,

Anderson-Bergman, Temple Lang, and Bodik 2017; nim 2025) under the chosen backend; SB uses Beta stick breaks in model code (see `dbeta`) and CRP uses the CRP distribution for allocations (see `dCRP`). Users control the number of iterations, burn-in, thinning, and number of chains through the `mcmc` argument in the wrappers.

Convergence checking is supported through standard multi-chain diagnostics and trace-based visualizations. The model object supports `summary()` for posterior summaries, `plot()` for diagnostic plots (trace, running mean, autocorrelation, and multi-chain diagnostics such as $\widehat{R}$ when multiple chains are used), and `params()` as a lightweight extractor returning posterior mean parameters reshaped to natural dimensions. Interval summaries used by `predict()` can be chosen as equal-tailed credible intervals or HPD intervals as described in Section 3.3 (Gelman, Carlin, Stern, Dunson, Vehtari, and Rubin 2013; Plummer, Best, Cowles, and Vines 2024). Diagnostic plotting relies on established MCMC diagnostic tooling (ggm 2025).

```r
print(fit)
p_hat <- params(fit)
p_hat
```

```
MixGPD fit | backend: Stick-Breaking Process | kernel:
Lognormal Distribution | GPD tail: TRUE
n = 100 | components = 5 | epsilon = 0.025
MCMC: niter=200, nburnin=50, thin=1, nchains=1
Fit
Use summary() for posterior summaries; plot() for
diagnostics; predict() for predictions.
Posterior mean parameters

$alpha
[1] 2.034

$w
[1] 0.4224 0.3063 0.1465 0.1158

$beta_meanlog
            x1      x2       x3
comp1  0.34970 -0.5504 -0.31760
comp2 -0.09937 -0.9372  0.10770
comp3  0.22010  0.5477  0.70350
comp4  0.04718  0.3581 -0.30940
comp5 -0.38810  0.8843 -0.04477

$sdlog
[1] 1.517 2.222 1.481 1.482

$beta_threshold
[1] -0.05201 -0.10530  0.07445

$sdlog_u
[1] 1.356

$beta_tail_scale
[1] 0.28030 0.04619 0.07101

$tail_shape
[1] 0.4271
```

Bulk-only special case (no GPD tail)

Setting `GPD=FALSE` removes the splicing step so that $f(y \mid x; \Theta) \equiv f_{\mathrm{DP}}(y \mid x; \Theta)$, and $L(\Theta, \Phi; \mathcal{D})$ reduces to the bulk-mixture likelihood in Θ. All estimation and prediction targets described below remain available, with tail-specific terms omitted. The wrapper `dpmix()` fits this bulk-only model, while `dpmgpd()` fits the augmented (spliced) model.

3.3. Posterior functionals, interval summaries and prediction

The estimands reported by CausalMixGPD are functionals of the estimated conditional distribution $F(\cdot \mid x)$ from Section 2. In this article we focus on four conditional functionals: density $f(y \mid x)$, survival $S(y \mid x) = 1 - F(y \mid x)$, quantiles $Q(\tau \mid x)$, and mean-type summaries.

Let $(\Theta^{(s)}, \Phi^{(s)})_{s=1}^{S}$ denote post-burn-in draws from the posterior. For density, survival, and quantiles, CausalMixGPD evaluates the corresponding draw-wise functional using the spliced formulas (14)–(17) (with the truncated bulk mixture), and then summarizes across draws. Specifically, for a generic scalar functional $T(\Theta, \Phi; x)$ (e.g., $f(y_0 \mid x)$ or $Q(\tau \mid x)$),

$$\widehat{T}(x) = \frac{1}{S} \sum_{s=1}^{S} T(\Theta^{(s)}, \Phi^{(s)}; x),$$

and uncertainty is summarized either by equal-tailed credible intervals (posterior quantiles) or by highest posterior density (HPD) intervals (shortest intervals covering the target posterior mass). Equal-tailed intervals and general Bayesian posterior summaries follow standard practice (Gelman *et al.* 2013); HPD intervals are computed from the MCMC draws using established implementations (Plummer *et al.* 2024).

Mean-type summaries are handled differently in the package. For `type="mean"` and `type="rmean"`, CausalMixGPD uses posterior predictive Monte Carlo within each posterior draw rather than direct plug-in evaluation. For a fixed covariate value x, within draw s the package generates

$$y_1^{(s)}, \ldots, y_M^{(s)} \sim f(\cdot \mid x; \Theta^{(s)}, \Phi^{(s)}),$$

and computes

$$\mu^{(s)}(x) = \frac{1}{M} \sum_{m=1}^{M} y_m^{(s)}, \qquad \mu_c^{(s)}(x) = \frac{1}{M} \sum_{m=1}^{M} \min\{y_m^{(s)}, c\},$$

corresponding to the conditional mean and the restricted mean at cutoff c, respectively. Posterior summaries and intervals are then formed from $\{\mu^{(s)}(x)\}_{s=1}^{S}$ or $\{\mu_c^{(s)}(x)\}_{s=1}^{S}$. Under the spliced model, the conditional mean is infinite when $\xi \geq 1$; accordingly, CausalMixGPD flags `type="mean"` as infinite when there is posterior mass with $\xi \geq 1$, while `type="rmean"` remains finite by construction.

Using the same fitted model `fit` from Section 3.1, we now compute posterior summaries on the original training covariates (default behavior of `predict()` when `newdata` is not supplied).

```
q_grid <- c(0.50, 0.90, 0.95, 0.99)
x_eval <- dat[1:20, c("x1", "x2", "x3")]

est_quant <- predict(fit, newdata = x_eval, type = "quantile", index = q_grid,
                     interval = "hpd", level = 0.95)
```

```
  id index  estimate     lower     upper
1  1  0.50  1.821068 0.5022973  2.909784
2  1  0.90  4.680770 3.1943038  6.411481
3  1  0.95  6.296861 4.0336562  9.313934
4  1  0.99 12.784902 7.2789201 23.359469
```

For a new covariate value $x^\star$, prediction is based on the posterior predictive distribution

$$p(y^\star \mid x^\star, \mathcal{D}) = \int f(y^\star \mid x^\star; \Theta, \Phi)\, p(\Theta, \Phi \mid \mathcal{D})\, d\Theta\, d\Phi.$$

The `predict()` method returns posterior summaries for the same functional targets as in Section 3.3. For `type="density"` and `type="survival"`, the user supplies a grid of y values and the method returns pointwise posterior means with interval bounds computed draw-by-draw. For `type="quantile"`, the user supplies quantile indices τ and the method returns posterior summaries of $Q(\tau \mid x^\star)$ computed using the spliced quantile representation in (17). For `type="mean"` and `type="rmean"`, prediction uses posterior predictive Monte Carlo as described above, with x replaced by $x^\star$ and with user-controlled M through `nsim_mean`.

```r
q_levels <- c(0.25, 0.50, 0.75)
x_new <- as.data.frame(lapply(
  dat[, c("x1", "x2", "x3")],
  quantile,
  probs = q_levels,
  na.rm = TRUE
))
rownames(x_new) <- c("q25", "q50", "q75")
y_grid <- seq(0, 10, length.out = 200)
p_grid <- c(0.50, 0.90, 0.95, 0.99)
```

```r
pdens <- predict(fit, newdata = x_new, y = y_grid, type = "density",
                 interval = "credible", level = 0.95)
```

```r
psurv <- predict(fit, newdata = x_new, y = y_grid, type = "survival",
                 interval = "credible", level = 0.95)
```

```r
pquant <- predict(fit, newdata = x_new, type = "quantile", index = p_grid,
                  interval = "hpd", level = 0.95)
```

```
             x1          x2            x3
q25 -0.59851252 -0.57151934 -0.7224762264
q50  0.02874559 -0.01658997  0.0004347721
q75  0.69727001  0.41546653  0.5516024827
[1] 0.00000000 0.05025126 0.10050251 0.15075377
[1] 0.50 0.90 0.95 0.99
```

```
  id          y       density         lower         upper
1  1 0.00000000 1.000000e-300 1.000000e-300 1.000000e-300
2  1 0.05025126  1.050943e+00  4.391043e-01  2.415552e+00
3  1 0.10050251  7.648774e-01  3.187806e-01  1.422371e+00
4  1 0.15075377  6.226713e-01  2.719930e-01  1.181847e+00
```

```
  id          y  survival     lower     upper
1  1 0.00000000 1.0000000 1.0000000 1.0000000
2  1 0.05025126 0.9055000 0.8106256 0.9673990
3  1 0.10050251 0.8613613 0.7228659 0.9408555
4  1 0.15075377 0.8266686 0.6671500 0.9196784
```

```
  id index estimate     lower     upper
1  1  0.50 1.482378 0.4926661  2.711455
2  1  0.90 3.924552 2.9765853  5.409868
3  1  0.95 5.090515 3.6744175  7.986932
4  1  0.99 9.792541 6.0698483 20.129287
```

3.4. Clustering extension

The mixture modeling framework introduced earlier induces a latent partition of the observations through the allocation variables z_i. While the primary modeling workflow of the package focuses on estimating conditional densities and causal functionals, the clustering extension instead treats this latent partition as the main object of interest. The package therefore provides a dedicated clustering interface that fits the same mixture models but returns clustering summaries derived from the posterior distribution of the allocation variables.

The clustering workflow mirrors the design used throughout the package. Users specify the response variable, optional covariates, the mixture kernel, and whether the model should use the bulk-only or the spliced likelihood. The resulting model is then fitted using the same MCMC framework used for the conditional and causal models. Consequently, clustering models share the same prior specification, kernel choices, and computational infrastructure as the rest of the package.

Two clustering wrappers are provided. The first constructs mixture models using only the bulk kernel distribution. The second uses the spliced bulk–tail likelihood introduced earlier, allowing the clustering structure to account for extreme observations through the generalized Pareto tail. Both interfaces accept the same model specification arguments as the main modeling functions, ensuring that clustering models can be constructed using the same syntax and options.

Three forms of covariate dependence are supported in clustering models. First, covariates may influence the mixture weights while the component distributions remain invariant across observations. In this configuration, clusters represent subpopulations whose prevalence varies with predictors. Second, the mixture weights may remain global while the component parameters depend on covariates through the same link structures used in the conditional density models. In this case clusters represent groups with distinct covariate-response relationships. Third, both the mixture weights and the component parameters may depend on covariates simultaneously, allowing clusters to capture both changing prevalence and covariate-dependent distributional behavior.

Posterior inference for clustering models uses the same sampling procedure as the main mixture models, jointly updating the component parameters and the allocation variables. Let $z^{(s)} = (z_1^{(s)}, \ldots, z_n^{(s)})$, $s = 1, \ldots, S$, denote the retained MCMC draws of the latent allocation vector after burn-in and thinning. Because mixture components are only identified up to label permutation, `CausalMixGPD` reports label-invariant clustering summaries based on pairwise co-clustering probabilities and a representative partition (Binder 1978; Dahl 2006).

The primary uncertainty summary is the posterior similarity matrix (PSM), computed entrywise as

$$\widehat{S}_{i\ell} = \frac{1}{S}\sum_{s=1}^{S} \mathbf{1}\Big\{z_i^{(s)} = z_\ell^{(s)}\Big\}, \qquad i,\ell = 1,\ldots,n.$$

Thus, $\widehat{S}_{i\ell}$ is the posterior probability that observations i and ℓ belong to the same cluster.

A single representative partition is then extracted by selecting the sampled partition whose adjacency matrix is closest to the PSM under squared loss (Dahl 2006):

$$s^\star = \arg\min_{1\le s\le S} \sum_{i=1}^{n}\sum_{\ell=1}^{n} \Big(\mathbf{1}\Big\{z_i^{(s)} = z_\ell^{(s)}\Big\} - \widehat{S}_{i\ell}\Big)^2.$$

The representative labels are $\hat{z}_i = z_i^{(s^\star)}$, relabeled to consecutive integers $1,\ldots,K$, and the reported cluster sizes are $n_k = \sum_{i=1}^{n} \mathbf{1}\{\hat{z}_i = k\}$.

When `return_scores = TRUE`, the package also returns a training membership score matrix. If $C_k = \{\ell : \hat{z}_\ell = k\}$ denotes cluster k in the representative partition, the unnormalized score for observation i and cluster k is

$$q_{ik} = \frac{1}{|C_k|}\sum_{\ell\in C_k} \widehat{S}_{i\ell},$$

and the normalized training score is

$$p_{ik} = \frac{q_{ik}}{\sum_{h=1}^{K} q_{ih}}.$$

The hard representative label is $\arg\max_{1\le k\le K} p_{ik}$, while a simple allocation-certainty summary is $\max_{1\le k\le K} p_{ik}$.

For a new observation $(y_i^{\text{new}}, x_i^{\text{new}})$, the package first forms within-draw component scores

$$r_{ij}^{(s)} \propto \omega_j^{(s)}(x_i^{\text{new}})\, f_j^{(s)}(y_i^{\text{new}} \mid x_i^{\text{new}}), \qquad j = 1,\ldots,J,$$

where $\omega_j^{(s)}(x)$ is the draw-specific gating weight when that quantity is available; otherwise the implementation falls back to sampled global weights, when present, or to the empirical component frequencies in draw s. For occupied components, these scores are mapped to representative clusters through

$$m_{jc}^{(s)} = \frac{1}{n_j^{(s)}} \sum_{\ell:\, z_\ell^{(s)}=j} \mathbf{1}\{\hat{z}_\ell = c\}, \qquad c = 1,\ldots,K,$$

and then averaged over posterior draws:

$$p_{ic}^{\text{new}} = \frac{1}{S}\sum_{s=1}^{S}\sum_{j=1}^{J} r_{ij}^{(s)} m_{jc}^{(s)}.$$

Empty sampled components are assigned a uniform map over the K representative clusters. These are the quantities returned internally by `predict.dpmixgpd_cluster_fit()`. After row normalization, the predicted cluster label is $\arg\max_{1\le c\le K} p_{ic}^{\text{new}}$.

The clustering workflow is illustrated below using one of the example data sets included with the package.

```
library(CausalMixGPD)

data("nc_real200_k2", package = "CausalMixGPD")
data("nc_realX100_p3_k2", package = "CausalMixGPD")

dat_cl <- data.frame(y = nc_realX100_p3_k2$y, nc_realX100_p3_k2$X)

fit_cluster <- dpmix.cluster(
  y ~ x1 + x2 + x3,
  data       = dat_cl,
  kernel     = "normal",
  type       = "weights",
  components = 8,
  mcmc = list(niter = 200, nburnin = 50, thin = 1, nchains = 1, seed = 11)
)
```

Posterior clustering summaries can be obtained directly from the fitted object. The following code returns the PSM, the representative training partition, the training cluster sizes, and the associated certainty scores.

```
cluster_psm   <- predict(fit_cluster, type = "psm")
cluster_train <- predict(fit_cluster, type = "label", return_scores = TRUE)

cluster_sizes     <- as.data.frame(table(cluster = cluster_train$labels))
cluster_certainty <- apply(cluster_train$scores, 1, max)
```

Cluster assignments for new observations are obtained from the posterior predictive score matrix in the same interface.

```
cluster_new <- predict(
  fit_cluster,
  newdata = dat_cl,
  type    = "label",
  return_scores = TRUE
)

cluster_new_certainty <- apply(cluster_new$scores, 1, max)
```

The same workflow applies to the spliced mixture model, which incorporates the generalized Pareto tail component when clustering data with heavy-tailed behavior.

```
fit_cluster_spliced <- dpmgpd.cluster(
  y ~ x1 + x2 + x3,
  data       = dat_cl,
  kernel     = "normal",
  type       = "both",
  components = 8,
  mcmc = list(niter = 200, nburnin = 50, thin = 1, nchains = 1, seed = 12)
)
```

3.5. Causal extension and propensity score augmentation

This package extends the one-arm conditional mixture regression framework to two-arm causal studies by fitting an outcome model under treatment and under control, optionally augmenting the outcome covariates with a PS summary. Throughout this section we write the model in its default *augmented spliced* form (bulk mixture + GPD tail), using the same conditional distributional building blocks introduced earlier (so we do not restate the spliced density; see (15) and the corresponding conditional functionals defined in the previous sections).

Data, model components, and likelihood factorization

Let

$$\mathcal{D} = \{(y_i, a_i, \boldsymbol{x}_i)\}_{i=1}^n, \qquad a_i \in \{0, 1\},$$

where $a_i = 1$ denotes treatment and $a_i = 0$ denotes control. The causal interface consists of two pieces.

First, when PS augmentation is enabled, a binary regression model is used for the treatment mechanism

$$a_i \mid \boldsymbol{x}_i, \boldsymbol{\gamma} \sim \text{Bernoulli}\big(e_{\boldsymbol{\gamma}}(\boldsymbol{x}_i)\big), \tag{51}$$

$$h(e_{\boldsymbol{\gamma}}(\boldsymbol{x}_i)) = \gamma_0 + \boldsymbol{x}_i^\top \boldsymbol{\gamma}, \qquad h \in \{\text{logit}, \text{probit}\}, \tag{52}$$

which is consistent with the definition and balancing property in (39)–(40). The package also supports a `"naive"` option, in which $e(\boldsymbol{x})$ is taken to be constant (no covariate dependence).

Second, the outcome model is fit separately within each arm. Denote by $F_a(\cdot \mid \boldsymbol{r})$ the fitted conditional outcome distribution under arm $a \in \{0, 1\}$, where $\boldsymbol{r}$ is the covariate vector used by the outcome model. With PS augmentation, the package constructs an *augmented* covariate vector

$$\boldsymbol{r}(\boldsymbol{x}) = \Big(1,\ \boldsymbol{x}^\top,\ \psi(\widehat{e}(\boldsymbol{x}))\Big)^\top, \tag{53}$$

where $\widehat{e}(\boldsymbol{x})$ is a posterior summary of the PS (posterior mean or median, controlled by `ps_summary`), and $\psi(\cdot)$ is either $\psi(p) = \text{logit}(p)$ or $\psi(p) = p$ (controlled by `ps_scale`). When PS augmentation is disabled, $\boldsymbol{r}(\boldsymbol{x}) = (1, \boldsymbol{x}^\top)^\top$.

Within each arm, the conditional density $f_a(y \mid \boldsymbol{r})$ follows the same mixture regression + optional GPD tail specification described in the one-arm section (kernel choice, link structure $g(\cdot)$ for covariate-dependent parameters, and either `"sb"` or `"crp"` backend for the DPM). Let $\boldsymbol{\Theta}_a$ collect the arm-specific mixture and tail parameters. The arm-wise likelihood contributions are

$$L_a(\boldsymbol{\Theta}_a; \mathcal{D}) = \prod_{i:\, a_i = a} f_a(y_i \mid \boldsymbol{r}(\boldsymbol{x}_i); \boldsymbol{\Theta}_a)\,, \qquad a \in \{0, 1\},$$

with $f_a(\cdot \mid \cdot)$ given by the spliced construction already defined (bulk mixture below threshold and GPD exceedances above threshold). Practically, the implementation uses a two-stage workflow: (i) fit the PS model using (51)–(52), then (ii) compute $\widehat{e}(\boldsymbol{x}_i)$ and build $\boldsymbol{r}(\boldsymbol{x}_i)$ in (53) before fitting the two arm-specific outcome models.

Prior specification (PS then outcome)

For the PS regression coefficients, the default prior is independent Normal,

$$\gamma_0 \sim \mathcal{N}(m_0, s_0^2), \qquad \gamma_k \sim \mathcal{N}(m_0, s_0^2),$$

with defaults controlled by `ps_prior`.

For each arm's outcome model, priors follow the same structure as the one-arm case: a DPM prior (with either truncated stick-breaking or CRP allocation, implemented in `nimble` (de Valpine *et al.* 2017; nim 2025)) and kernel-/tail-specific priors on the bulk and GPD parameters. Since the causal fit is simply a pair of one-arm fits (treated and control), all customization options for kernels, links, and tail inclusion apply arm-wise.

MCMC algorithm, diagnostics, and parameter extraction

The causal fit runs MCMC in two stages, each with its own iteration controls.

(i) PS stage: posterior simulation for $\boldsymbol{\gamma}$ in (51)–(52) using `mcmc_ps` settings. The PS posterior draws are summarized to obtain $\widehat{e}(\boldsymbol{x}_i)$, which is optionally transformed via `ps_scale` and appended to the outcome covariates as in (53).

(ii) Outcome stage: arm-specific posterior simulation for $\boldsymbol{\Theta}_0$ and $\boldsymbol{\Theta}_1$ using `mcmc_outcome` settings, with either `"sb"` or `"crp"` backend.

For routine convergence checks, users can inspect trace plots and autocorrelation for monitored parameters and (when running multiple chains) compare between-chain and within-chain mixing. The package stores MCMC samples in the fitted object, and standard diagnostics can be applied after converting samples to `coda` objects (e.g., effective sample size, Geweke diagnostics, and HPD intervals via `coda::HPDinterval`; see the `coda` reference at the end of this message). Posterior parameter summaries can be extracted via the exported `params()` method, which returns arm-specific parameter tables for causal fits.

Estimation and interval summaries for causal contrasts

All reported causal quantities are computed as functionals of the fitted arm-specific conditional distributions. Let $T_a(\boldsymbol{x})$ denote any conditional functional under arm a (density, survival, quantile, or mean/restricted mean), computed from $F_a(\cdot \mid \boldsymbol{r}(\boldsymbol{x}))$ using the same definitions as in the one-arm case. The corresponding conditional causal contrast is

$$\Delta_T(\boldsymbol{x}) = T_1(\boldsymbol{x}) - T_0(\boldsymbol{x}).$$

The package computes $\Delta_T(\boldsymbol{x})$ *draw-by-draw* from posterior samples, then summarizes $\Delta_T(\boldsymbol{x})$ by a posterior mean/median and uncertainty intervals. Equal-tailed credible intervals are formed by posterior quantiles, while HPD intervals are computed using `coda::HPDinterval`.

The main causal estimators provided by the package are:

(a) Conditional effects at covariate profiles. For a user-supplied set of covariate rows $\{\boldsymbol{x}_m^\star\}_{m=1}^M$,

$$\mathrm{CATE}(\boldsymbol{x}_m^\star) = \mu_1(\boldsymbol{x}_m^\star) - \mu_0(\boldsymbol{x}_m^\star), \qquad \mathrm{CQTE}(\tau \mid \boldsymbol{x}_m^\star) = Q_1(\tau \mid \boldsymbol{x}_m^\star) - Q_0(\tau \mid \boldsymbol{x}_m^\star),$$

where $\mu_a(\boldsymbol{x}) = \mathbb{E}(Y \mid A = a, \boldsymbol{X} = \boldsymbol{x})$ and $Q_a(\tau \mid \boldsymbol{x})$ is the conditional quantile function. In heavy-tailed settings, $\mu_a(\boldsymbol{x})$ can be unstable when the implied tail shape yields infinite mean; the package therefore supports both `type="mean"` and `type="rmean"` for (restricted) mean-based effects, with restricted means computed as $\mathbb{E}[\min\{Y, c\} \mid A = a, \boldsymbol{X} = \boldsymbol{x}]$ at user-specified cutoff c.

(b) Marginal (standardized) summaries. The package reports marginal ATE and marginal

QTE-style summaries by empirical standardization over covariate profiles:

$$\mathrm{ATE} = \frac{1}{n}\sum_{i=1}^{n}\Big\{\mu_1(\boldsymbol{x}_i) - \mu_0(\boldsymbol{x}_i)\Big\}, \tag{54}$$

$$\mathrm{QTE}(\tau) = Q_1^{\mathrm{m}}(\tau) - Q_0^{\mathrm{m}}(\tau), \qquad Q_a^{\mathrm{m}}(\tau) = \inf\{y : F_a^{\mathrm{m}}(y) \geq \tau\}, \quad F_a^{\mathrm{m}}(y) = \frac{1}{n}\sum_{i=1}^{n} F_a(y \mid \boldsymbol{x}_i). \tag{55}$$

Treated-standardized analogues (ATT, QTT) are obtained by replacing the full-sample covariate empirical distribution with the treated-subsample empirical distribution $\{i : a_i = 1\}$. In the current implementation, `ate()` and `att()` aggregate draw-wise conditional mean contrasts across rows, while `qte()` and `qtt()` compute draw-wise arm-specific marginal quantiles and then contrast them.

Posterior prediction at new covariate profiles

Prediction in the causal setup proceeds by producing arm-specific predictive summaries at new covariates and then differencing when required. For new covariates $\boldsymbol{x}^\star$, the package first predicts the PS (if enabled), forms $\boldsymbol{r}(\boldsymbol{x}^\star)$ via (53), and then evaluates the desired distributional functional under both arms. For `type = "mean"` and `type = "quantile"`, `predict()` returns treated-minus-control summaries and stores arm-specific predictions as attributes; for `type = "density"`, `"survival"`, and `"prob"`, it returns arm-specific summaries directly.

A recommended way to build an interpretable prediction grid is to use coordinate-wise empirical quartiles. Let $q_{k,p}$ be the pth sample quantile of covariate x_k for $p \in \{0.25, 0.5, 0.75\}$. The grid

$$\mathcal{X}_\star = \prod_{k=1}^{p}\{q_{k,0.25}, q_{k,0.5}, q_{k,0.75}\}$$

yields 3^p representative covariate profiles at which CATE, CQTE, survival contrasts, and other predictive contrasts can be reported.

User-facing workflow (code)

The following example uses the formula interface and the high-level wrapper, fitting the augmented spliced model by default.

```
data("causal_pos500_p3_k2", package = "CausalMixGPD")
dat <- causal_pos500_p3_k2
df  <- data.frame(y = dat$y, A = dat$A, dat$X)
```

```
mcmc_fixed <- list(
  niter = 200,
  nburnin = 50,
  thin = 1,
  nchains = 1,
  seed = 1
)
```

```
cfit <- dpmgpd.causal(
  formula = y ~ x1 + x2 + x3,
  data = df,
  treat = "A",
  backend = "sb",
  kernel = "lognormal",
  components = 5,
  PS = "logit",
  ps_scale = "logit",
  ps_summary = "mean",
  mcmc_outcome = mcmc_fixed,
  mcmc_ps = mcmc_fixed
)
```

```
CausalMixGPD causal fit
PS model: Bayesian logit (A | X)
Outcome (treated): backend = sb | kernel = lognormal
Outcome (control): backend = sb | kernel = lognormal
GPD tail (treated/control): TRUE / TRUE
```

```
ate_hat <- ate(cfit, level = 0.90, interval = "credible")
```

```
  id estimate     lower    upper
1  1 133.6313 0.5208869 789.4111
```

```
qte_hat <- qte(cfit, probs = c(0.50, 0.90, 0.95), level = 0.90, interval = "hpd")
```

```
  id index  estimate       lower     upper
1  1  0.50 -0.230319 -0.72616380 0.2434466
2  1  0.90  1.581025  0.04824207 2.8446616
3  1  0.95  3.424051 -1.11525657 7.0265043
```

```
Xnew <- df[1:5, c("x1","x2","x3")]
```

```
cate_hat <- cate(cfit, newdata = Xnew, type = "mean", level = 0.90)
```

```
  id    estimate      lower    upper
1  1  0.15628140 -0.8192142 1.208809
2  2 -0.01701753 -1.8635060 2.979435
3  3  0.80363038 -0.2959428 2.721403
4  4 -0.19121860 -1.7339494 2.143942
```

```
cqte_hat <- cqte(cfit, newdata = Xnew, probs = c(0.50, 0.90), level = 0.90)
```

```
  id index    estimate      lower     upper
1  1   0.5  0.09848583 -0.3263803 0.5465127
2  1   0.9  0.14929438 -0.9261457 1.4969607
3  2   0.5 -0.03731305 -0.4291908 0.3870325
4  2   0.9 -0.14538125 -1.8493190 2.0484655
```

```
qs    <- c(0.25, 0.50, 0.75)
Xgrid <- expand.grid(lapply(df[c("x1","x2","x3")], quantile, probs = qs))
```

```
pred_q <- predict(cfit, newdata = Xgrid, type = "quantile", p = c(0.50, 0.90))
```

```
data frame with 0 columns and 0 rows
```

```
pred_s <- predict(cfit, newdata = Xgrid, type = "survival", y = rep(4, nrow(Xgrid)))
```

```
data frame with 0 columns and 0 rows
```

```
pred_q_trt <- attr(pred_q, "trt")
pred_q_con <- attr(pred_q, "con")
```

```
  id index  estimate     lower     upper
1  1   0.5 1.0871813 0.5465243 2.3651155
2  1   0.9 3.1515644 1.7912098 4.8641579
3  2   0.5 0.4514065 0.2045529 0.8192195
4  2   0.9 1.6172941 0.9007377 2.9213755
  id index  estimate     lower     upper
1  1   0.5 1.5998935 0.9799795 2.0270002
2  1   0.9 3.5348458 2.9220149 4.3121050
3  2   0.5 0.4242338 0.2666140 0.7774886
4  2   0.9 1.4014587 0.9884698 2.2199421
```

Bulk-only causal mixtures (no GPD tail)

To run the same two-arm regression without a GPD tail, use the bulk-only wrapper `dpmix.causal()` (or set `GPD=FALSE` at bundle construction). All causal estimand and prediction functions (`ate`, `qte`, `cate`, `cqte`, and `predict`) operate the same way, now interpreting $F_a(\cdot \mid \boldsymbol{x})$ as the bulk mixture distribution.

4. Data analysis I: clustering under a bulk-only conditional mixture

This application demonstrates the clustering wing for a conditional mixture model under a bulk-only likelihood (no GPD tail). The objective is to infer latent groups of observational units implied by the fitted mixture model, and to summarize posterior clustering uncertainty in a label-invariant way. Throughout, clustering is reported via (i) the posterior similarity

matrix (PSM) and (ii) a single representative partition derived from the posterior draws, together with out-of-sample cluster label prediction for new units.

4.1. Dataset and objective

We use the `Boston` housing data (MASS), treating each census tract as an observational unit. Let

$$\mathcal{D} = \{(y_i, \boldsymbol{x}_i)\}_{i=1}^n,$$

where y_i is the per-capita crime rate (`crim`) for tract i, and $\boldsymbol{x}_i$ is a vector of tract characteristics. Our aim is twofold: (i) to cluster tracts using a flexible conditional mixture model that can accommodate skewness and heterogeneity, and (ii) to assign cluster labels to new tracts using posterior predictive co-clustering evidence.

4.2. Preprocessing and train/test split

We split the data into a training set used to fit the clustering model and a test set used only for cluster-label prediction. Covariates are standardized using training-set moments to avoid leakage.

```
library(CausalMixGPD)
library(MASS)

set.seed(123)
data("Boston", package = "MASS")

dat <- Boston
dat <- subset(dat, select = c(crim, lstat, rm, nox))

n <- nrow(dat)
idx_train <- sample(seq_len(n), size = floor(0.75 * n), replace = FALSE)
train_dat <- dat[idx_train, ]
test_dat  <- dat[-idx_train, ]

# standardize covariates using training set
xvars <- c("lstat", "rm", "nox")
mu_x  <- vapply(train_dat[, xvars], mean, numeric(1))
sd_x  <- vapply(train_dat[, xvars], sd,   numeric(1))

train_dat[, xvars] <- sweep(sweep(train_dat[, xvars], 2, mu_x, "-"), 2, sd_x, "/")
test_dat[, xvars]  <- sweep(sweep(test_dat[, xvars],  2, mu_x, "-"), 2, sd_x, "/")
```

```
  sample   n
1  train 379
2   test 127
```

4.3. Clustering model specification and fit

We fit a bulk-only conditional mixture model for `crim` with three covariates. In this illustration, we select `type="param"`, meaning covariates enter through component-specific regression parameters while the mixture weights are global. This configuration clusters observations by

Table 1: Training cluster allocation (counts)

sample	C1	C2	C3	C4	C5	C6	C7
train	55	62	160	14	76	9	3

their regression behavior and distributional differences while keeping cluster prevalence interpretable as a global mixing proportion.

The fitted object is subsequently summarized through posterior co-clustering probabilities (PSM) and a representative partition.

```
mcmc_fixed <- list(
  niter = 2000,
  nburnin = 500,
  thin  = 1,
  nchains = 1,
  seed = 123
)

fit_clust <- dpmix.cluster(
  formula    = crim ~ lstat + rm + nox,
  data       = train_dat,
  kernel     = "lognormal",
  type       = "weight",
  components = 20,
  mcmc       = mcmc_fixed
)
```

4.4. Posterior similarity matrix and representative partition

Because mixture components are not identifiable up to permutation, clustering summaries are reported using label-invariant quantities. The PSM is defined by

$$S_{i\ell} = \Pr(z_i = z_\ell \mid \text{data}), \qquad i, \ell = 1, \ldots, n,$$

and is estimated by averaging co-clustering indicators across posterior draws. A single representative partition is then extracted from the posterior draws by selecting a sampled partition whose induced adjacency structure is closest to the PSM under squared loss, yielding stable cluster labels for the training data.

The following commands extract the training PSM and the representative labels.

```
PSM_train <- predict(fit_clust, type = "psm")
z_train   <- predict(fit_clust, type = "label")
```

For presentation we report the number of clusters and cluster sizes under the representative partition (Table 1).

A visualization of the PSM provides a global view of co-clustering uncertainty. Block structure indicates groups of tracts that are consistently assigned together across posterior draws, as shown in Figure 3, “Posterior similarity matrix for the training sample.”

```
image(PSM_train$psm, main = "Posterior similarity matrix (training)")
```

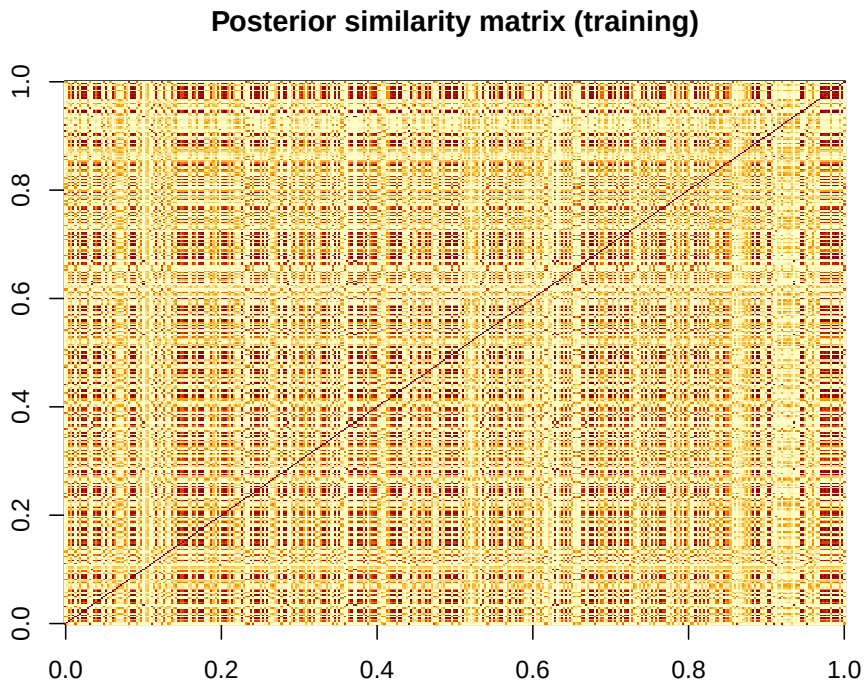

Figure 3: Posterior similarity matrix for the training sample.

```
PSM dimension: 379 x 379
```

4.5. Cluster labels for new observations

Cluster assignment for new observations is performed through the same prediction interface. The output is a label in the space of the representative training clusters, obtained by aggregating posterior predictive co-clustering evidence at the cluster level.

```
z_test <- predict(
  fit_clust,
  newdata = test_dat,
  type    = "label"
)
```

To compare training and new-sample clustering structure, we tabulate the representative cluster sizes in the training sample and the predicted cluster sizes in the test sample (Tables 2 and 3).

Table 2: Train allocation (horizontal)

sample	C1	C2	C3	C4	C5	C6	C7
train	55	62	160	14	76	9	3

Table 3: Test allocation (horizontal)

sample	C1	C2	C3	C4	C5	C6
test	1	32	63	6	23	2

We also visualize the shift in cluster frequencies from training to test to assess whether the inferred grouping structure generalizes; see Figure 4, captioned "Cluster frequencies in training and test samples under representative labels."

```
ggplot(cluster_old_new_counts, aes(x = cluster, y = n, fill = sample)) +
  geom_col(position = "dodge") +
  labs(
    title = "Cluster frequencies: training vs test",
    x = "Cluster (representative partition)",
    y = "Count",
    fill = "Sample"
  ) +
  theme_minimal(base_size = 10)
```

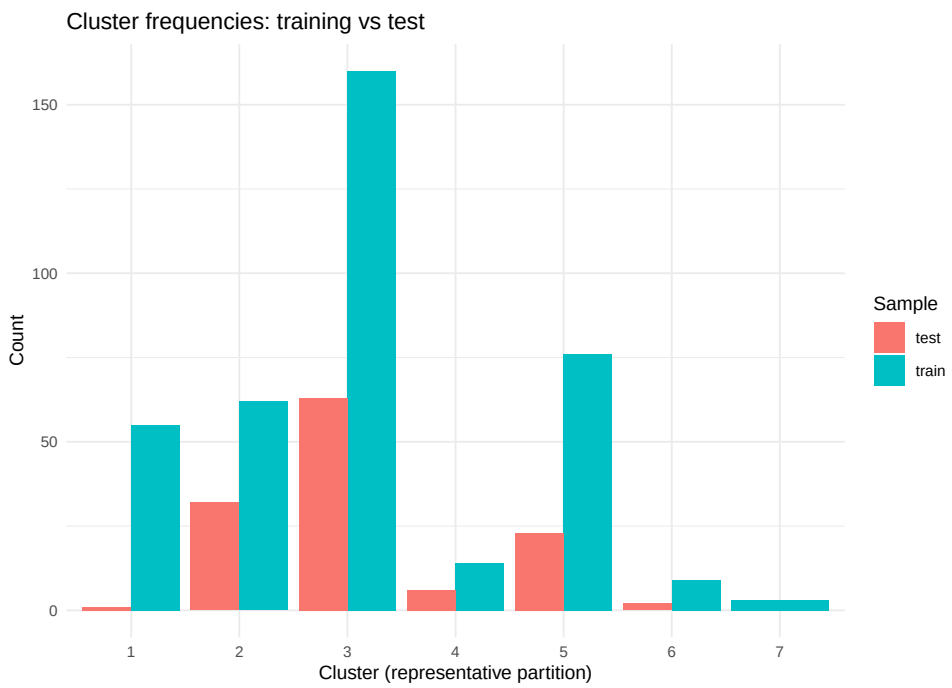

Figure 4: Cluster frequencies in training and test samples under representative labels.

```
Bar plot rendered above.
```

This analysis illustrates the intended clustering workflow: inference is summarized through co-clustering probabilities and a representative training partition, and new observations are assigned to those clusters using posterior predictive co-clustering evidence. In Section 5, we next turn to the causal workflow, which targets treatment contrasts rather than latent partitions.

Data analysis II: Spliced causal mixture regression with ATE, QTE, CATE and CQ

This application demonstrates causal estimands from a two-arm spliced DPM regression with a GPD tail. In contrast to Section 4, where the focus is model-based clustering under a CRP bulk-only regression, here the focus is inference on mean and quantile treatment contrasts, including conditional (covariate-specific) effects and extreme upper-tail behavior.

5.1. Dataset and objective

The AER package does not ship the National JTPA experimental data as an in-package dataset. Instead, we use a bundled causal benchmark data object to provide a clean and reproducible entry point for a heavy-tailed earnings-style application.

Let Y denote post-program earnings (USD), $A \in \{0, 1\}$ denote randomized assignment to training, and $\boldsymbol{x}$ denote baseline covariates including previous earnings and demographics. The scientific goal is to quantify how training changes (i) average earnings and (ii) distributional features of earnings, with special attention to the upper tail.

Throughout, we report four causal estimands:

$$\begin{aligned}
\mathrm{ATE} &= \mathbb{E}\{Y(1)\} - \mathbb{E}\{Y(0)\}, \\
\mathrm{QTE}(\tau) &= Q_{Y(1)}(\tau) - Q_{Y(0)}(\tau), \\
\mathrm{CATE}(\boldsymbol{x}) &= \mathbb{E}\{Y(1) \mid \boldsymbol{X} = \boldsymbol{x}\} - \mathbb{E}\{Y(0) \mid \boldsymbol{X} = \boldsymbol{x}\}, \\
\mathrm{CQTE}(\tau, \boldsymbol{x}) &= Q_{Y(1)|\boldsymbol{X}=\boldsymbol{x}}(\tau) - Q_{Y(0)|\boldsymbol{X}=\boldsymbol{x}}(\tau),
\end{aligned}$$

where $Q(\tau)$ denotes the τ-quantile. The standardized estimands (ATE, QTE) integrate over the empirical distribution of $\{\boldsymbol{x}_i\}_{i=1}^n$; the conditional estimands (CATE, CQTE) are evaluated at user-chosen covariate profiles.

5.2. Spliced DPM regression model

I model arm-specific conditional outcome distributions $F_a(\cdot \mid \boldsymbol{x})$ via a spliced construction:

$$f_a(y \mid \boldsymbol{x}) = \sum_j w_{aj}(\boldsymbol{x})\, k(y \mid \Theta_{aj})\, \mathbf{1}(y \le u) + \{1 - F_a(u \mid \boldsymbol{x})\}\, g(y \mid u, \sigma_a, \xi_a)\, \mathbf{1}(y > u),$$

where the bulk kernel $k(\cdot)$ is a DPM regression (stick-breaking backend) and the tail density $g(\cdot)$ is GPD with scale $\sigma_a > 0$ and shape ξ_a. The GPD tail index ξ_a controls extremal heaviness (e.g., $\xi_a > 0$ corresponds to heavy tails). This model stabilizes inference for extreme quantiles by coupling nonparametric bulk flexibility with EVT-consistent tail extrapolation.

5.3. Model fitting

I fit a two-arm spliced mixture using a lognormal bulk kernel. The MCMC settings below are intentionally modest for vignette speed; in substantive analysis we recommend longer runs and sensitivity checks (e.g., components, threshold priors, and prior scales).

```
mcmc_fixed <- list(
  niter = 120,
  nburnin = 40,
  thin = 1,
  nchains = 1,
  seed = 101,
  show_progress = FALSE
)
```

```
fit <- dpmgpd.causal(
  x = y,
  X = X,
  treat = A,
  backend = "sb",
  kernel = "normal",
  components = 4,
  mcmc = mcmc_fixed
)
```

```
     metric value
1         n   500
2         p     3
3 n_treated   309
4 n_control   191
```

5.4. Standardized estimands: ATE and QTE

I first summarize the overall average effect using `ate()`, which aggregates draw-wise conditional mean contrasts over the empirical covariate distribution. I then compute quantile effects at a grid including upper-tail levels.

```
ate_fit <- ate(fit, show_progress = FALSE)
```

```
  id   estimate      lower      upper
1  1 -0.4121421 -0.7170418 0.09078951
```

```
qte_fit <- qte(fit, probs = taus, show_progress = FALSE)
```

```
  id index   estimate       lower      upper
1  1  0.25 -0.8197706 -1.48164947 -0.2415758
2  1  0.50 -0.6946461 -1.51414175 -0.1011650
3  1  0.75  0.7299151 -0.25027110  1.2937407
4  1  0.90  0.7794739 -0.08451773  1.6423832
```

```
ate_effect_plot <- plot(ate_fit, type = "effect")
ate_effect_plot
```

```
qte_effect_plot <- plot(qte_fit, type = "effect")
qte_effect_plot
```

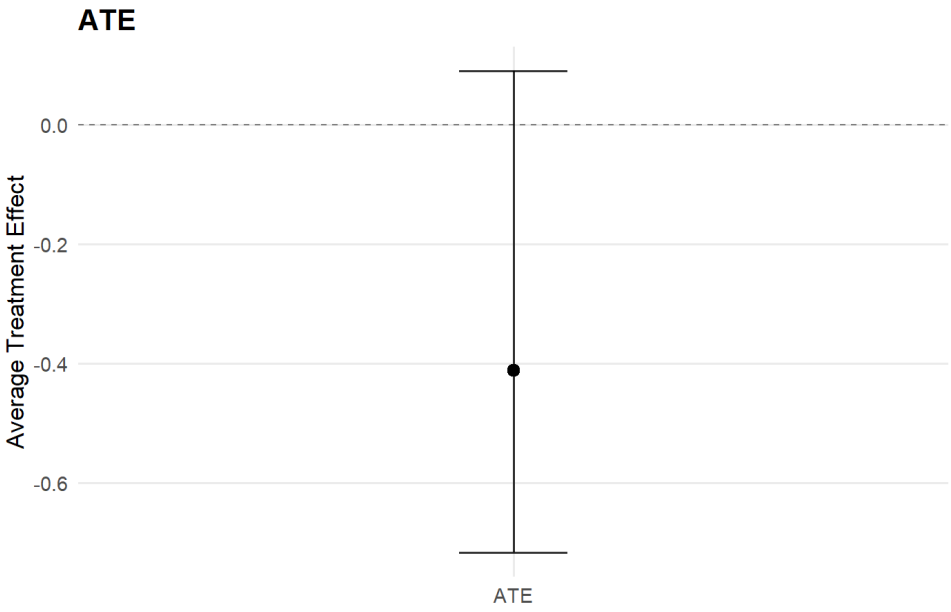

Figure 5: Posterior ATE effect summary for the spliced causal model.

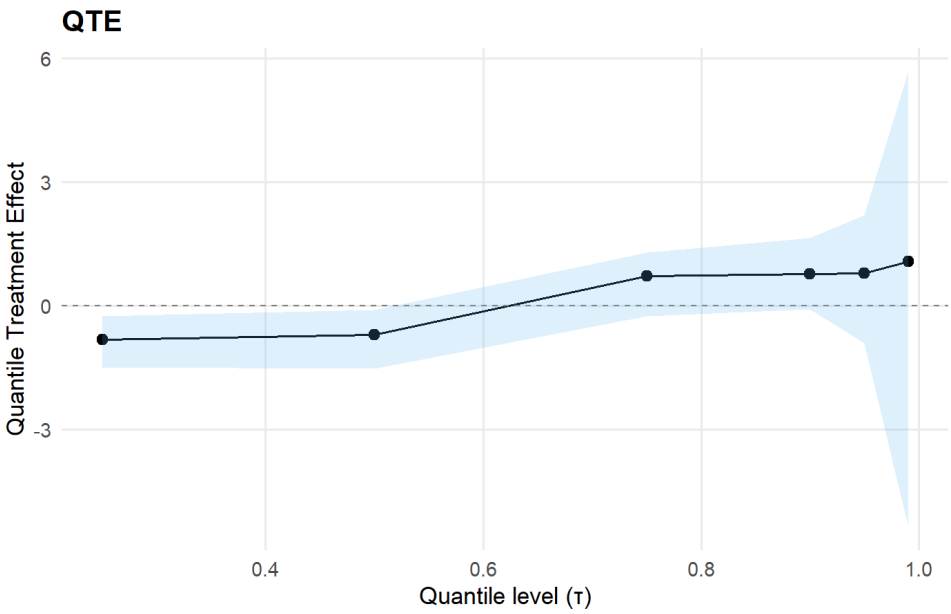

Figure 6: Posterior QTE effect summary across quantile levels.

The marginal treatment-effect visual summaries are shown in Figure 5, captioned "Posterior ATE effect summary for the spliced causal model," for the ATE, and Figure 6, captioned "Posterior QTE effect summary across quantile levels." for the QTE.

5.5. Conditional estimands: CATE and CQTE at representative profiles

To illustrate heterogeneity, we evaluate conditional effects at two covariate profiles: (i) a 'typical" profile at sample medians and (ii) a'high previous earnings" profile at the 80th percentile of previous earnings, holding other covariates at medians.

```
col_med <- apply(X, 2, median, na.rm = TRUE)
x_typical <- col_med
x_highprev <- col_med
profile_var <- if ("prevearn" %in% colnames(X)) "prevearn" else colnames(X)[1]
x_highprev[profile_var] <- as.numeric(quantile(X[, profile_var], 0.80, na.rm = TRUE))
xnew <- rbind(typical = x_typical, high_prev_earn = x_highprev)
```

```
  profile_var
1          x1
                       x1          x2         x3
typical        0.07649836 -0.09628321 0.01209002
high_prev_earn 0.79678466 -0.09628321 0.01209002
```

```
cate_fit <- cate(fit, newdata = xnew, show_progress = FALSE)
```

```
  id   estimate      lower      upper
1  1 -0.3645759 -0.6971991  0.1087821
2  2 -1.4191524 -2.2463081 -0.6754342
```

```
cqte_fit <- cqte(fit, probs = taus, newdata = xnew, show_progress = FALSE)
```

```
  id index   estimate      lower       upper
1  1  0.25 -0.2157592 -0.5805429  0.08519506
2  1  0.50 -0.2653653 -0.4044642 -0.07230213
3  1  0.75 -0.2282270 -0.4720426  0.05270702
4  1  0.90 -0.1783537 -0.8925062  0.33589134
```

CATE/CQTE effect plots are omitted in this PDF.

5.6. Tail behavior

A key advantage of the spliced model is direct posterior inference on the tail index ξ_a by arm. Larger ξ_a indicates a heavier upper tail and greater propensity for extreme earnings.

```
        arm      mean         sd    q0.025    q0.500
23  treated 0.5830602 0.04488664 0.4780895 0.5805598
231 control 0.3667860 0.11154470 0.1951404 0.4457394
       q0.975
23  0.6883067
231 0.4635703
```

5.7. Interpretation

The ATE summarizes the average shift in earnings under training, while QTE describes how the effect varies across the marginal outcome distribution. CATE and CQTE extend these contrasts to covariate-specific comparisons, illustrating treatment effect heterogeneity in both central and extreme quantiles. Finally, arm-specific tail indices ξ_a provide a parsimonious summary of whether training changes extremal behavior beyond what is captured by bulk-only regression.

6. Discussion

7. Supplementary Materials

References

(2023). *BCEE: The Bayesian Causal Effect Estimation Algorithm*. doi:10.32614/CRAN.package.BCEE. R package on CRAN, URL https://cran.r-project.org/package=BCEE.

(2023). *dirichletprocess: Build Dirichlet Process Objects for Bayesian Modelling*. doi:10.32614/CRAN.package.dirichletprocess. R package on CRAN, URL https://cran.r-project.org/package=dirichletprocess.

(2024). *bcf: Causal Inference using Bayesian Causal Forests*. doi:10.32614/CRAN.package.bcf. R package on CRAN, URL https://cran.r-project.org/package=bcf.

(2024). *POT: Generalized Pareto Distribution and Peaks Over Threshold*. doi:10.32614/CRAN.package.POT. R package on CRAN, URL https://cran.r-project.org/package=POT.

(2025). *bartCause: Causal Inference using Bayesian Additive Regression Trees*. doi:10.32614/CRAN.package.bartCause. R package on CRAN, URL https://cran.r-project.org/package=bartCause.

(2025). *evdbayes: Bayesian Analysis in Extreme Value Theory*. doi:10.32614/CRAN.package.evdbayes. R package on CRAN, URL https://cran.r-project.org/package=evdbayes.

(2025). *ggmcmc: Analysis of MCMC Samples and Bayesian Inference*. doi:10.32614/CRAN.package.ggmcmc. R package on CRAN, URL https://cran.r-project.org/package=ggmcmc.

(2025). *nimble: MCMC, Particle Filtering, and Programmable Hierarchical Modeling*. doi:10.32614/CRAN.package.nimble. R package on CRAN, URL https://cran.r-project.org/package=nimble.

Balkema AA, de Haan L (1974). "Residual Life Time at Great Age." *The Annals of Probability*, **2**(5), 792–804. doi:10.1214/aop/1176996548.

Behrens CN, Lopes HF, Gamerman D (2004). "Bayesian Analysis of Extreme Events with Threshold Estimation." *Statistical Modelling*, **4**(3), 227–244. doi:10.1191/1471082X04st075oa.

Binder DA (1978). "Bayesian Cluster Analysis." *Biometrika*, **65**(1), 31–38. doi:10.1093/biomet/65.1.31.

Blackwell D, MacQueen JB (1973). "Ferguson Distributions Via Pólya Urn Schemes." *The Annals of Statistics*, **1**(2), 353–355. doi:10.1214/aos/1176342372.

Coles S (2001). *An Introduction to Statistical Modeling of Extreme Values*. Springer, London. doi:10.1007/978-1-4471-3675-0.

Corradin R, Canale A, Nipoti B (2021). "BNPmix: An R Package for Bayesian Nonparametric Modeling via Pitman–Yor Mixtures." *Journal of Statistical Software*, **100**(15), 1–47. doi:10.18637/jss.v100.i15.

Dahl DB (2006). *Model-Based Clustering for Expression Data via a Dirichlet Process Mixture Model*. Cambridge University Press. In: *Bayesian Inference for Gene Expression and Proteomics*, Kim-Anh Do, Peter Müller, Marina Vannucci (Eds.).

De Iorio M, Müller P, Rosner GL, MacEachern SN (2004). "An ANOVA Model for Dependent Random Measures." *Journal of the American Statistical Association*, **99**(465), 205–215. doi:10.1198/016214504000000205.

de Valpine P, Turek D, Paciorek CJ, Anderson-Bergman C, Temple Lang D, Bodik R (2017). "Programming with Models: Writing Statistical Algorithms for General Model Structures with NIMBLE." *Journal of Statistical Software*, **76**(10), 1–27. doi:10.18637/jss.v076.i10.

Durrieu G, Grama I, Jaunatre K, Pham QK, Tricot JM (2018). "extremefit: A Package for Extreme Quantiles." *Journal of Statistical Software*, **87**(12), 1–20. doi:10.18637/jss.v087.i12.

Escobar MD, West M (1995). "Bayesian Density Estimation and Inference Using Mixtures." *Journal of the American Statistical Association*, **90**(430), 577–588. doi:10.1080/01621459.1995.10476550.

Ferguson TS (1973). "A Bayesian Analysis of Some Nonparametric Problems." *The Annals of Statistics*, **1**(2), 209–230. doi:10.1214/aos/1176342360.

Firpo S (2007). "Efficient Semiparametric Estimation of Quantile Treatment Effects." *Econometrica*, **75**(1), 259–276. doi:10.1111/j.1468-0262.2007.00738.x.

Frigessi A, Haug O, Rue H (2002). "A Dynamic Mixture Model for Unsupervised Tail Estimation without Threshold Selection." *Extremes*, **5**(3), 219–235. doi:10.1023/A:1024072610684.

Gelman A, Carlin JB, Stern HS, Dunson DB, Vehtari A, Rubin DB (2013). *Bayesian Data Analysis*. 3 edition. Chapman and Hall/CRC, Boca Raton, FL. doi:10.1201/b16018.

Hill JL (2011). "Bayesian Nonparametric Modeling for Causal Inference." *Journal of Computational and Graphical Statistics*, **20**(1), 217–240. doi:10.1198/jcgs.2010.08162.

Imbens GW, Rubin DB (2015). *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press, Cambridge.

Jara A, Hanson T, Quintana F, Müller P, Rosner G (2011). "DPpackage: Bayesian Semi- and Nonparametric Modeling in R." *Journal of Statistical Software*, **40**(5), 1–30. doi:10.18637/jss.v040.i05.

Lo AY (1984). "On a Class of Bayesian Nonparametric Estimates: I. Density Estimates." *The Annals of Statistics*, **12**(1), 351–357. doi:10.1214/aos/1176346412.

MacEachern SN (1999). *Dependent Nonparametric Processes*. ASA Proceedings of the Section on Bayesian Statistical Science, URL https://u.osu.edu/maceachern.1/files/2025/05/1999-MacEachern-JSM-Proceedings.pdf.

MacEachern SN (2000). *Dependent Dirichlet Processes*. Technical Report, Department of Statistics, The Ohio State University, URL https://u.osu.edu/maceachern.1/files/2025/05/2000-MacEachern-DDP-Tech-Report.pdf.

Neal RM (2000). "Markov Chain Sampling Methods for Dirichlet Process Mixture Models." *Journal of Computational and Graphical Statistics*, **9**(2), 249–265. doi:10.1080/10618600.2000.10474879.

Neyman J (1990). "On the Application of Probability Theory to Agricultural Experiments. Essay on Principles. Section 9." *Statistical Science*, **5**(4), 465–480.

Pickands J (1975). "Statistical Inference Using Extreme Order Statistics." *The Annals of Statistics*, **3**(1), 119–131. doi:10.1214/aos/1176343003.

Plummer M, Best N, Cowles K, Vines K (2024). *coda: Output Analysis and Diagnostics for MCMC*. R package version 0.19-4.1, URL https://cran.r-project.org/package=coda.

Quintana FA, Müller P, Jara A, MacEachern SN (2022). "The Dependent Dirichlet Process and Related Models." *Statistical Science*, **37**(1), 24–49. doi:10.1214/20-STS819.

Ren L, Du L, Carin L, Dunson DB (2011). "The Logistic Stick-Breaking Process." *Journal of Machine Learning Research*, **12**(7), 203–239. URL https://www.jmlr.org/papers/v12/ren11a.html.

Rosenbaum PR, Rubin DB (1983). "The Central Role of the Propensity Score in Observational Studies for Causal Effects." *Biometrika*, **70**(1), 41–55. doi:10.1093/biomet/70.1.41.

Rubin DB (1974). "Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies." *Journal of Educational Psychology*, **66**(5), 688–701. doi:10.1037/h0037350.

Sethuraman J (1994). "A Constructive Definition of Dirichlet Priors." *Statistica Sinica*, **4**(2), 639–650.

Talbot D, Lefebvre G, Atherton J, Chiu Y (2015). "The Bayesian Causal Effect Estimation Algorithm." *Journal of Causal Inference*, **3**(2), 207–236. doi:10.1515/jci-2014-0035.

Affiliation:

Givenname Familyname
Affiliation
Address, Country
E-mail: name@address
URL: https://link/to/webpage/

Journal of Statistical Software https://www.jstatsoft.org/
published by the Foundation for Open Access Statistics https://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd